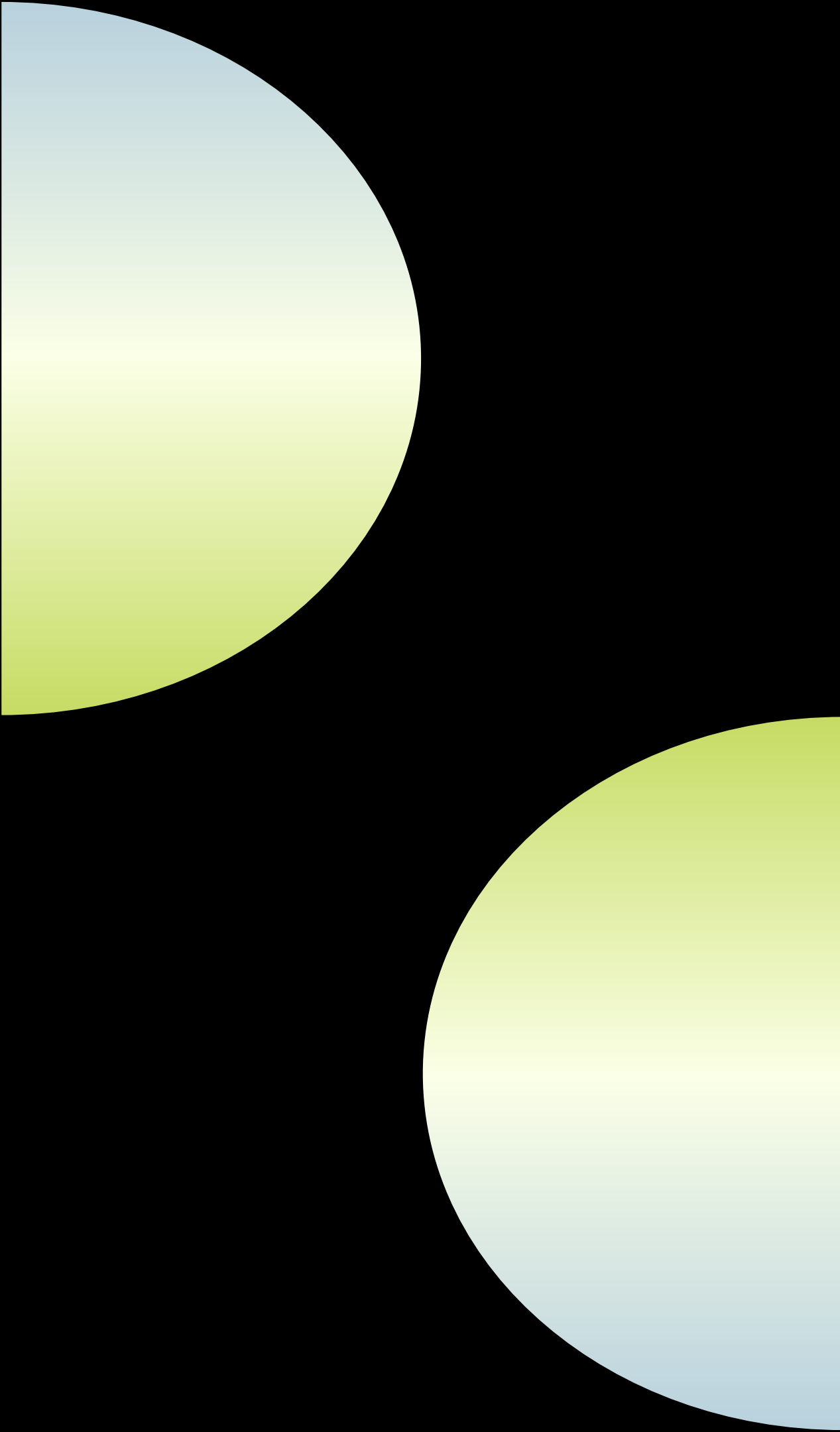




HPML Final Project: Scaling LLM Inference Under High Concurrency

Dev Goyal, Alessandro Castillo,
Anoushka Khajanchi

COMS 6998



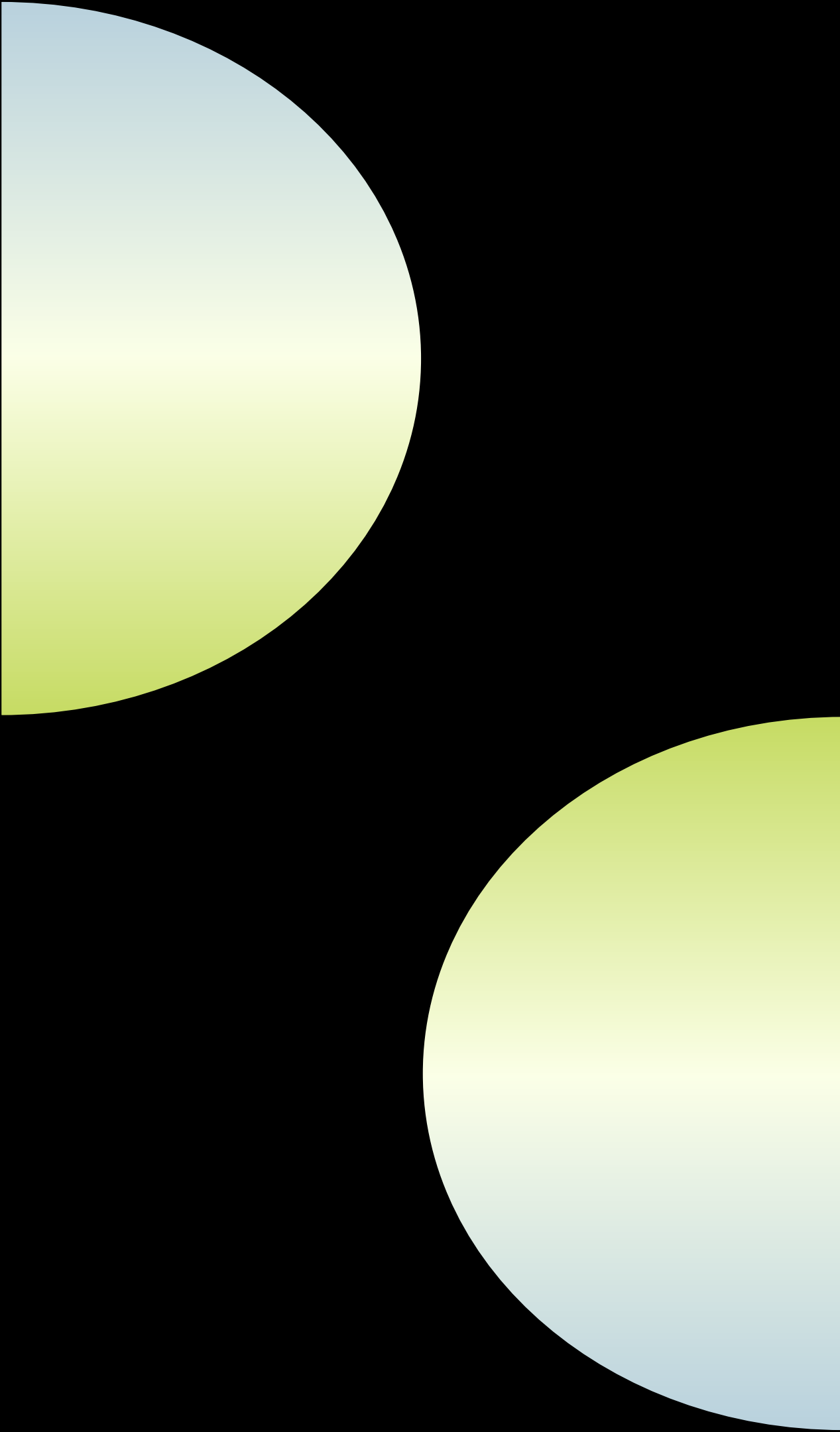
Introduction

Paradigm Shift

- LLM usage is evolving from single requests → highly concurrent workloads
- Many simultaneous requests share large context windows (e.g., codebases, system prompts)

Project Objective

- Benchmark a concurrent code-review workload under constrained VRAM
- Evaluate modern inference optimizations (e.g., better batching, memory sharing)
- Investigate how to:
 - a. Prevents OOM failures
 - b. Improves throughput & latency
 - c. Reduces cost of serving LLMs at scale



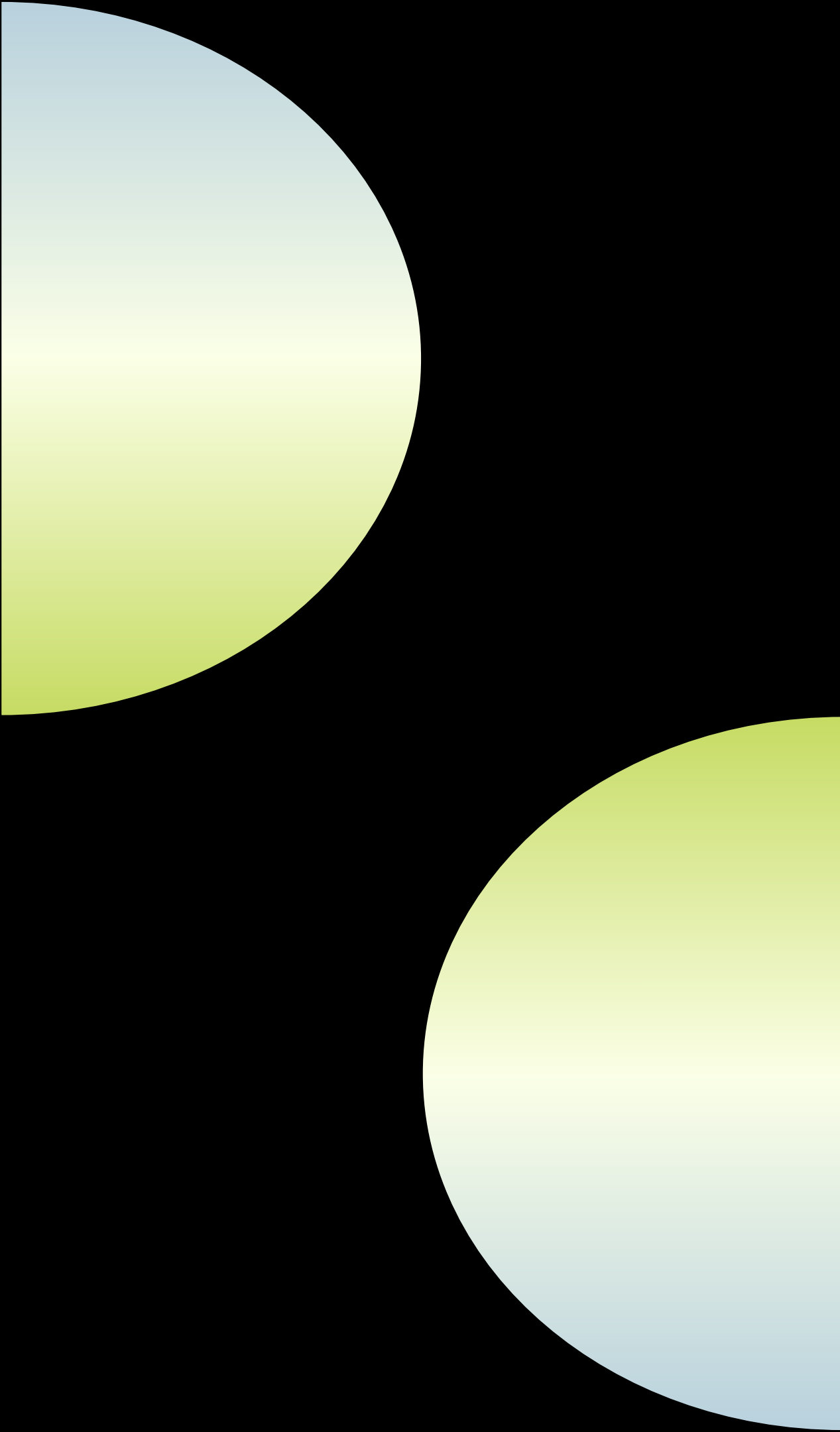
Hardware & Resources

GPU

- L4 GPU on Google Cloud
- 24GB memory

Target Model

- HuggingFace Qwen-7B-Instruct Agentic AI model
- Vary number of subagents used in the model proportionally with number of needed concurrent LLM calls



Metrics Recorded

Agents (Success / OOM)

- Number of concurrent requests completed vs failed
- Indicates system reliability under load (OOM = out-of-memory crash)

TTFT (Time To First Token)

- Time from request → first generated token
- Measures initial latency / responsiveness
- p99 = worst-case tail latency

Throughput (tok/s)

- Tokens generated per second across the system
- Measures overall system efficiency and capacity

VRAM Peak

- Maximum GPU memory used during inference
- Reflects memory efficiency and scaling limits

Wall Time

- Total time to complete all requests
- Captures end-to-end system performance under concurrency

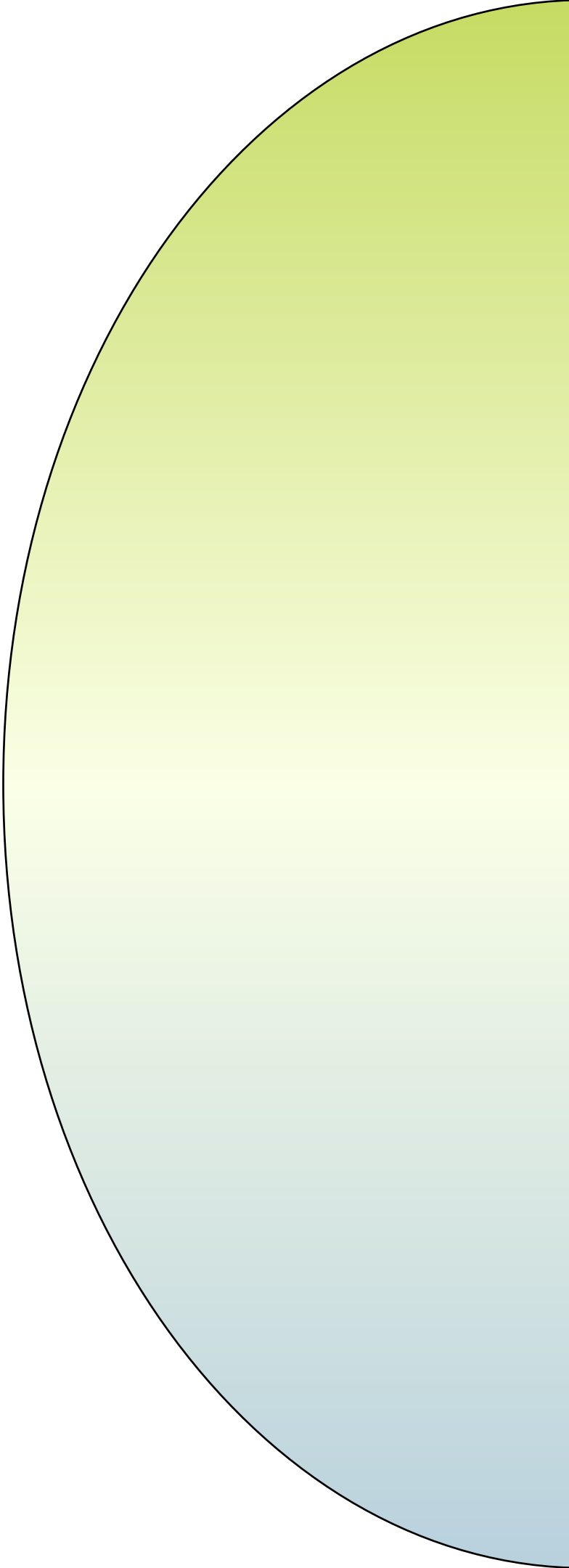
Baseline Results

Concuccurecy	Success	Throughput (tok/s)	TTFT avg (p99)	VRAM Peak	Wall Time
1	1/1	14.3	0.02s	19.24GB	19.6s
15	11/15	0.9	0.07s	21.57 GB	311.8s
50	8/50	1.2	0.08s	22.20 GB	236.1s

Key Takeaways:

- 14.3 → 0.9 tok/s (~93.75% drop)
- Adding concurrency → worse performance
- VRAM nearly saturated (~22GB / 24GB)
- OOMs spike: 0 → 42 failures
- System becomes memory-bound

Perplexity remains roughly stable (agent avg and base code ~ 2.0), thus the degradation is systems-driven, not model quality





Optimizations:

❖ **Optimization 1: Quantization**

- Reduces model precision (e.g., FP16 → INT8 / NF4) to compress weights and KV cache, lowering memory usage
- Enables running larger models or more concurrent requests by trading numerical precision for efficiency

❖ **Optimization 2: Speculative Decoding**

- Uses a smaller “draft” model to predict multiple tokens ahead, which are then verified by the main model
- Speeds up generation by reducing the number of expensive forward passes of the full model

❖ **Optimization 3: vLLM**

- An inference engine that uses continuous batching and optimized KV cache management to serve many requests efficiently
- Dynamically schedules and interleaves requests to maximize GPU utilization and throughput

Optimization 1: Quantization

Concuccurecy	Success	Throughput (tok/s)	TTFT avg (p99)	VRAM Peak	Wall Time
1	1/1	6.5	0.04s	9.56 GB	41.9s
15	15/15	0.3	0.04s	16.37Gb	1005.7s
50	17/50	0.3	0.10s	19.12Gb	828.4s

Key Takeaways:

- VRAM nearly halved at low concurrency (9.56GB vs ~19GB baseline)
- Enables 0 OOM at 15 concurrent requests
- 6.5 → 0.3 tok/s (~20× drop)
- Wall time explodes (1000s+)
- Quantization introduces compute + latency overhead

Comparison vs Baseline (bf16)

- Memory vs Stability Tradeoff
 - NF4 avoids OOM at medium concurrency (15/15 vs 11/15)
 - But still fails at scale (33 OOM at 50)
- Throughput Tradeoff
 - Baseline faster at all scales (e.g., 14.3 vs 6.5 tok/s at 1 request)
 - Quantization worsens throughput collapse under concurrency

Optimization 2: Speculative Decoding

Concuccurecy	Success	Throughput (tok/s)	TTFT avg (p99)	VRAM Peak	Wall Time
1	1/1	9.0	0.02s	20.23 GB	30.0s
15	6/15	1.0	0.06s	21.93Gb	273.6s
50	3/50	2.3	0.04s	21.10Gb	123.7s

Key Takeaways:

- As the number of agents needed for concurrent calls increases we see the Throughput go down initially dramatically then starts to increase again
- Wall time inversely increases dramatically at 15 agents then decreases at 50 agents.
- Number of successful calls peaks at 15 agents

Comparison vs Baseline (bf16)

- VRAM remains around 10% higher than baseline runs
- Throughput is around 30% lower than baseline at 15 agents, but 40%-80% higher than baseline at 1 and 50 agents.
- Wall time is only lower than baseline at 15 agents
- Number of successful agents remains lower than baseline at 15 and 50 agents

Optimization 3: VLLM

Concuccurecy	Success	Throughput (tok/s)	TTFT avg (p99)	Wall Time
1	1/1	15.2	1.64s	16.8s
15	15/15	10.6	2.06s	24.5s
50	50/50	9.1	3.18s	31.7s

Key Takeaways:

- 15.2 → 9.1 tok/s (only ~1.7× drop from 1 → 50)
- Contrast: baseline drops ~16×
- 0 OOM at all concurrency levels
- Efficient KV cache management + batching
- We don't have VRAM Peak metrics because it is vLLM-managed
- ~1.6–3.2s vs milliseconds in baseline
- Due to scheduling + batching overhead
- vLLM converts a memory-bound system → throughput-optimized system

Comparison vs Baseline (bf16)

- Scalability Breakthrough
 - Baseline: 8/50 succeed (42 OOM)
 - vLLM: 50/50 succeed (0 OOM)
- Throughput Preservation
 - Baseline collapses to ~1 tok/s
 - vLLM sustains ~9–10 tok/s under load

VLLM Metrics Continued:

Concuccurecy	Success	Throughput (tok/s)	TTFT avg (p99)	Wall Time
100	100/100	7.9	5.42s	37.7s
200	200/200	5.7	19.64s	74.37s
400	400/400	3.7	48.48s	159.0s

Key Takeaways

- 7.9 → 3.7 tok/s (100 → 400)
- No catastrophic collapse (unlike baseline)
- Latency Becomes the Bottleneck → TTFT grows significantly: 5.4s → 48.5s avg
- Queueing + scheduling dominate at high load
- 0 OOM across 400 concurrent requests
- Confirms effective KV cache + memory management
- System remains stable and efficient even under extreme load

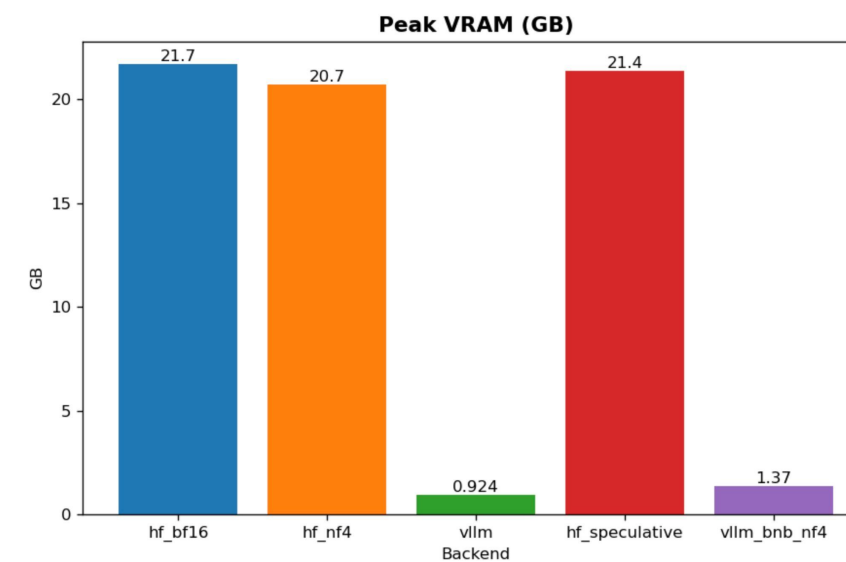
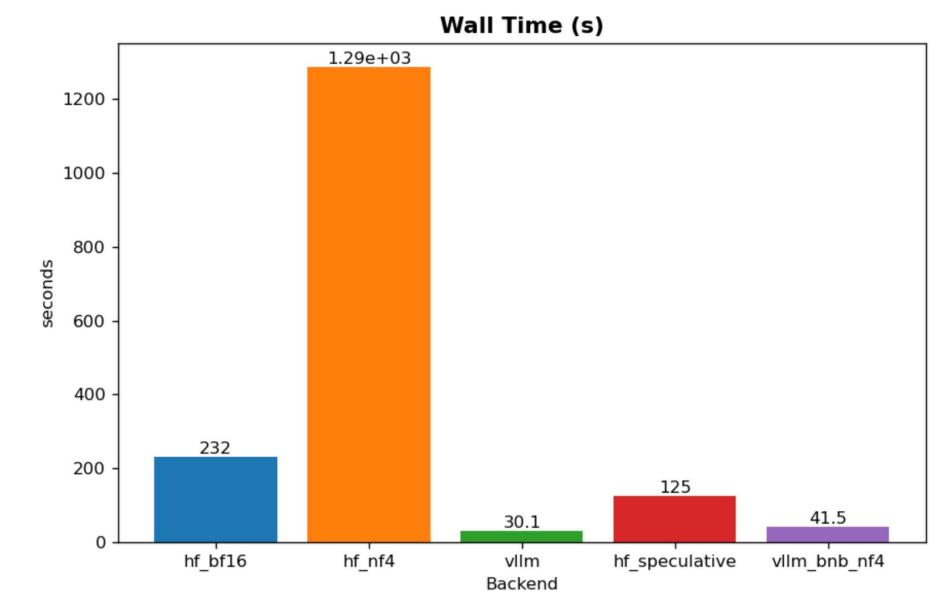
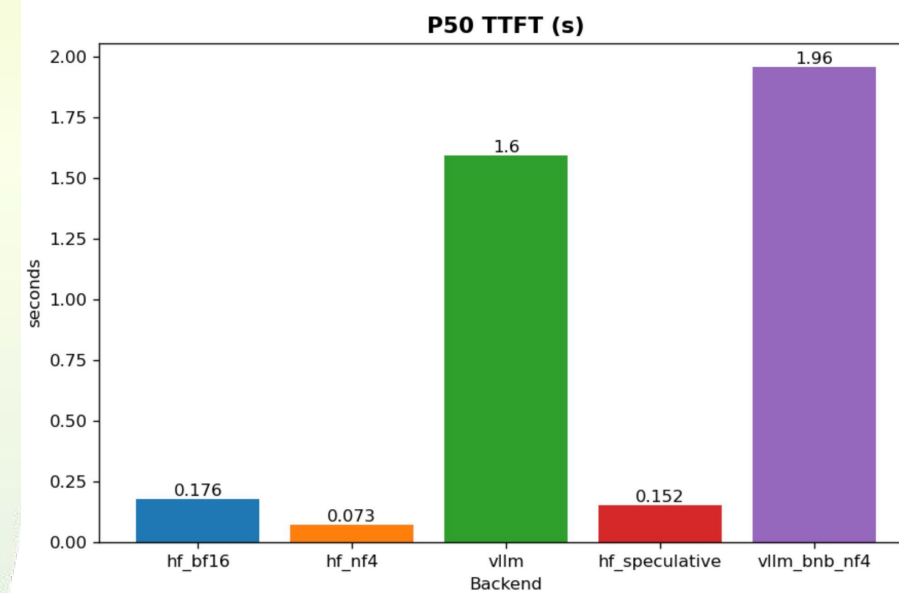
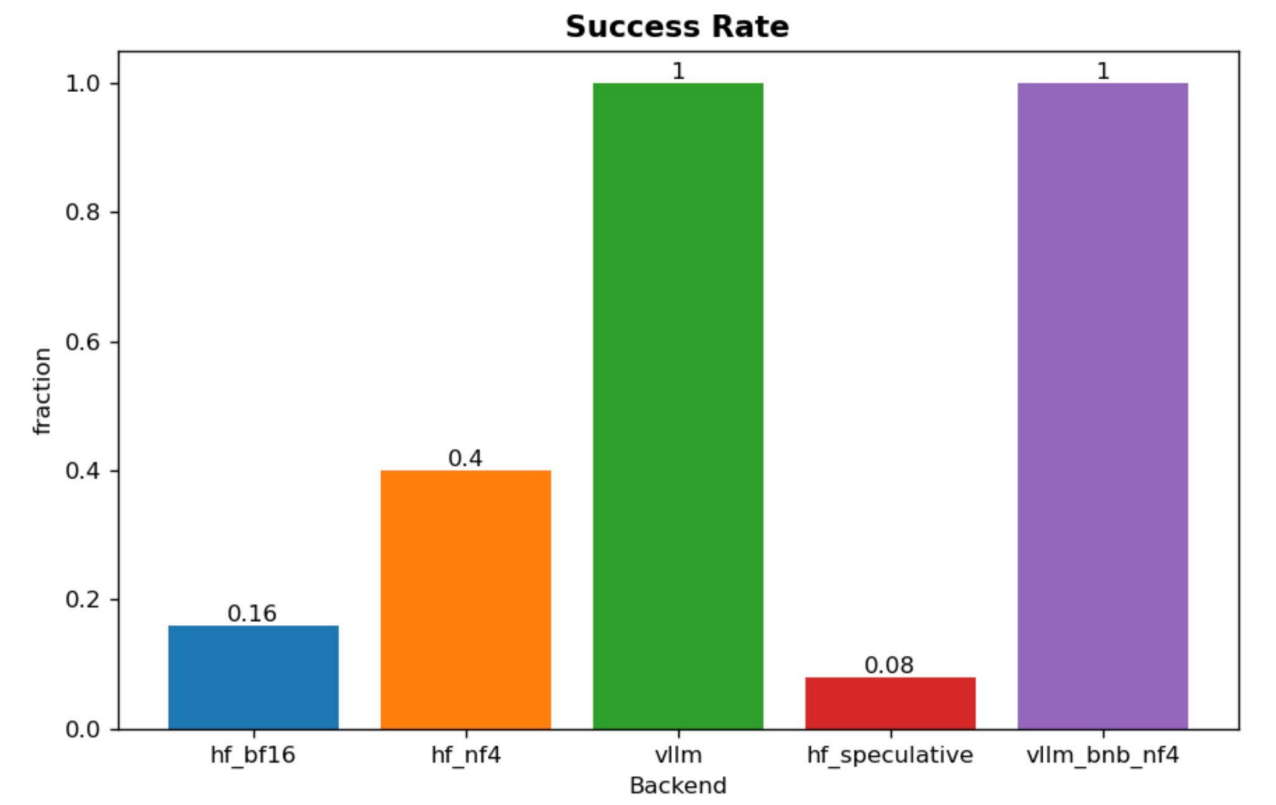
Summarization of Initial Results

Quantization helps but it slows down the processes.

Speculative decoding didn't outperform baseline metrics. The smaller model failed often which increased overhead because the larger model would have it to correct.

vLLM was the best optimization. It reduces OOM errors and is fast.

Findings and Results



Combining VLLM + Quantization Results

Concuccurecy	Success	Throughput (tok/s)	TTFT avg (p99)	Wall Time
1	1/1	11.3	2.05	22.7s
50	50/50	3.9	1.95s	69.1s
400	400/400	2.9	57.0	192.4s

What Changes with Quantization

- Reduces model memory footprint
 - Enables more aggressive packing of model weights + KV cache pressure relief
- Shifts tradeoff toward compute/latency overhead
 - Slight throughput degradation compared to pure vLLM
- Does not change vLLM's core scaling behavior
 - Still fully stable (0 OOM up to 400 concurrency)

Key Takeaway: quantization doesn't improve VLLM too much, the only potential improvement is miniscule and with memory

Single Kernel Analysis: NYS-REP Files

Kernel Category	HF BF16	vLLM	vLLM+NF4
GEMV decode (gemvx)	73.4%	78.2%	11.1%
INT4 decode (kgemm_4bit_naive)	0.0%	0.0%	40.6%
Prefill GEMM (CUTLASS)	12.2%	14.6%	26.1%
Unfused activations	3.2%	0.0%	0.0%
FlashAttention	0.7%	3.3%	5.9%

Profiling limitation: HF NF4 + Speculative traces unavailable due to nsys v2023.4.4 qdstrm import bug (known NVIDIA defect)

GEMV Decode (gemvx)

- Matrix–vector multiply used during token-by-token decoding
- Happens after the first token, when sequence grows → memory-bound step

INT4 Decode (kgemm_4bit_naive)

- Matrix multiply using 4-bit quantized weights during decoding
- Reduces memory usage but adds dequantization + compute overhead

Prefill GEMM (CUTLASS)

- Large matrix–matrix multiply used during the prefill phase (processing the full prompt)
- Highly optimized (via NVIDIA CUTLASS) → compute-heavy, very efficient

Unfused Activations

- Activation functions (e.g., GELU, ReLU) computed as separate kernels
- Less efficient than fused ops → extra memory reads/writes

FlashAttention

- Optimized attention mechanism that reduces memory movement
- Avoids materializing full attention matrices → faster + more memory-efficient

Baseline: What GPU Is Doing

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
64.9	66513445562	179780	369971.3	526462.0	5216	4423158	421974.3	std::enable_if<!T7, void>::type internal::gemv::kernel<int, int,
8.6	8861407373	2258	3924449.7	2861096.5	601758	40292438	4027336.2	void cutlass::Kernel2<cutlass 80 tensorop bf16 s16816gemm_relu bf1
8.5	8709473847	71400	121981.4	117727.5	103711	215712	17641.0	std::enable_if<!T7, void>::type internal::gemv::kernel<int, int,
3.6	3641706461	644	5654823.7	4770419.5	1773914	13935190	3190391.9	ampere bf16 s16816gemm bf16 256x128 ldg8 f2f stages 32x3 tn
2.6	2713980054	71400	38010.9	33024.0	21952	98496	12463.9	void at::native::<unnamed>::CatArrayBatchedCopy_vectorized<at::na
1.3	1308849855	109019	12005.7	1792.0	1152	2798232	102699.9	void at::native::vectorized_elementwise_kernel<(int)4, at::native
1.2	1201577672	146687	8191.4	2400.0	1824	486398	22682.7	void at::native::elementwise_kernel<(int)128, (int)4, void at::na
0.9	925110330	144088	6420.5	1600.0	928	630334	22879.6	void at::native::vectorized_elementwise_kernel<(int)4, at::native
0.7	759903389	35700	21285.8	17984.0	16064	93728	6341.4	void pytorch_flash::flash_fwd_splitkv_kernel<Flash_fwd_kernel_tra
0.7	743560228	76582	9709.3	2784.0	1312	8972904	109642.9	void at::native::unrolled_elementwise_kernel<at::native::direct co
0.7	723757708	36344	19914.1	5344.0	1568	1804795	113827.3	void at::native::vectorized_elementwise_kernel<(int)4, at::native
0.7	691434943	73986	9345.5	1824.0	1535	763262	42029.9	void at::native::elementwise_kernel<(int)128, (int)2, void at::na
0.6	648143200	73976	8761.5	3360.0	1952	1613788	35380.7	void at::native::<unnamed>::CatArrayBatchedCopy<at::native::<unna
0.6	624007280	73986	8434.1	3168.0	2272	619582	28929.3	void at::native::reduce_kernel<(int)512, (int)1, at::native::Reduc
0.6	566319703	308	1838700.3	288287.0	130527	5259376	1770569.3	void pytorch_flash::flash_fwd_kernel<Flash_fwd_kernel_traits<(int)
0.5	555228147	73986	7504.5	1313.0	1088	708765	40260.1	void at::native::vectorized_elementwise_kernel<(int)4, void at::na
0.5	484588929	76582	6327.7	1312.0	1087	553598	29550.3	void at::native::vectorized_elementwise_kernel<(int)4, at::native
0.4	426448662	35700	11945.3	6272.0	5664	120799	12453.1	void pytorch_flash::flash_fwd_splitkv_combine_kernel<Flash_fwd_ke
0.4	388254131	72688	5341.4	2240.0	1504	269119	9997.1	void at::native::elementwise_kernel<(int)128, (int)4, void at::na
0.4	362230756	1400	258736.3	167743.5	83935	894397	180388.8	ampere bf16 s1688gemm bf16 128x128 ldg8 relu f2f stages 32x1 tn

gemvx dominates at 73.4%. Unfused silu + rsqrt + reduce = 3.2% wasted on separate kernel launches.

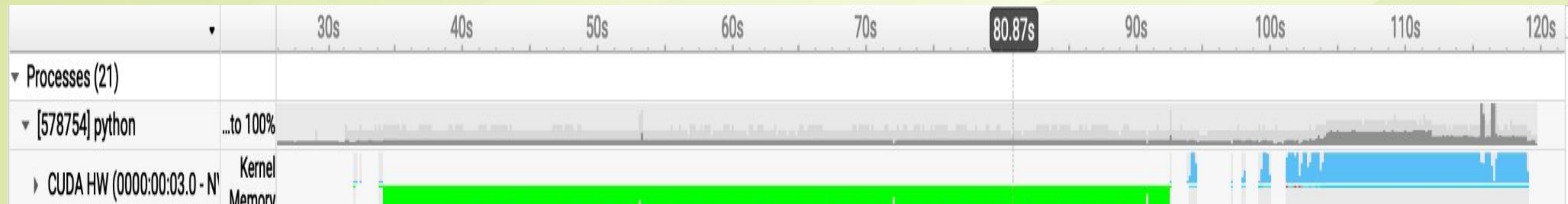


Timeline — 5 agents. Long H2D memory transfer phase (~60s) precedes compute. Blue decode region shows CUDA kernel serialization between agents.

vLLM: What GPU Is Doing

Time	Total Time	Instances	Avg	Med	Min	Max	StdDev	Name
51.4%	9.343 s	21420	436.172 µs	134.495 µs	104.096 µs	1.087 ms	446.687 µs	std::enable_if<!T7, void>::type internal::gemvx::kernel<int, int, __nv_bfloat16, __nv_bfloat16, __nv_bfloat16, float, (bool)0, (bool)1,
26.8%	4.864 s	7401	657.217 µs	524.990 µs	522.238 µs	4.303 ms	691.566 µs	std::enable_if<!T7, void>::type internal::gemvx::kernel<int, int, __nv_bfloat16, __nv_bfloat16, __nv_bfloat16, float, (bool)0, (bool)1,
10.3%	1.880 s	509	3.693 ms	1.151 ms	554.238 µs	37.418 ms	4.058 ms	void cutlass::Kernel2<cutlass_80_tensorop_bf16_s16816gemm_relu_bf16_256x128_32x3_tn_align8>(T1::Params)
4.3%	772.655 ms	196	3.942 ms	4.646 ms	1.657 ms	4.987 ms	1.128 ms	ampere_bf16_s16816gemm_bf16_256x128_ldg8_f2f_stages_32x3_tn
1.9%	342.188 ms	7140	47.925 µs	47.999 µs	44.640 µs	50.239 µs	713 ns	void flash::flash_fwd_splitkv_kernel<Flash_fwd_kernel_traits<(int)128, (int)64, (int)128, (int)4, (bool)0, (bool)0, cutlass::bfloat16_
1.4%	254.972 ms	168	1.518 ms	1.524 ms	530.398 µs	2.409 ms	587.522 µs	void flash::flash_fwd_splitkv_kernel<Flash_fwd_kernel_traits<(int)128, (int)64, (int)128, (int)4, (bool)0, (bool)0, cutlass::bfloat16_
1.0%	183.906 ms	7364	24.973 µs	5.664 µs	3.935 µs	894.269 µs	123.310 µs	void vllm::act_and_mul_kernel<c10::BFloat16, __nv_bfloat162, &vllm::silu_kernel<c10::BFloat16>, &vllm::packed_silu_kernel<__nv
0.5%	89.836 ms	14728	6.099 µs	3.488 µs	2.495 µs	133.759 µs	17.294 µs	std::enable_if<T2>(int)0&&vllm::typeConvert<T1>::exists, void>::type vllm::fused_add_rms_norm_kernel<c10::BFloat16, (int)8>('
0.4%	67.927 ms	7140	9.513 µs	9.569 µs	6.688 µs	10.592 µs	480 ns	void flash::flash_fwd_splitkv_combine_kernel<Flash_fwd_kernel_traits<(int)128, (int)64, (int)128, (int)4, (bool)0, (bool)0, cutlass::
0.3%	52.918 ms	261	202.752 µs	50.720 µs	35.296 µs	16.617 ms	1.502 ms	void at::native::<unnamed>::cunn_SoftMaxForward<(int)4, float, float, float, at::native::<unnamed>::LogSoftMaxForwardEpilogue
0.3%	46.953 ms	12	3.913 ms	4.835 ms	1.991 ms	4.936 ms	1.400 ms	void at::native::mbtopk::computeBlockDigitCounts<float, unsigned int, unsigned int, (int)2>(at::cuda::detail::TensorInfo<const T1
0.2%	38.974 ms	7364	5.292 µs	3.712 µs	2.624 µs	82.432 µs	10.348 µs	void vllm::rotary_embedding_kernel<c10::BFloat16, (bool)1>(const long *, T1 *, T1 *, const T1 *, int, long, long, long, int, int, int)

gemvx still 78.2% but unfused ops are GONE.
PagedAttention ops appear: reshape_cache +
slot_mapping.

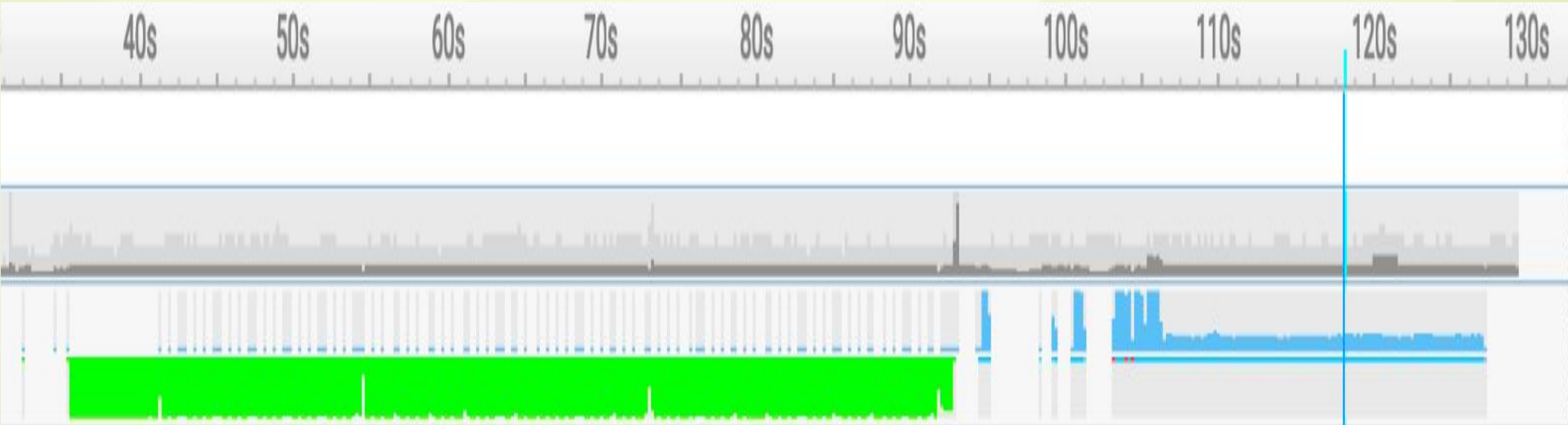


Timeline — 1 agent. Extended green phase = vLLM pre-allocating PagedAttention KV pool. Blue decode blocks are more consolidated than HF baseline.

vLLM+NF4: Quantization Cost

Time	Total Time	Instances	Avg	Med	Min	Max	StdDev	Name
40.6%	4.058 s	49980	81.190 µs	33.984 µs	8.257 µs	218.271 µs	69.870 µs	void kgemm_4bit_inference_naive<__nv_bfloat16, (int)128, (int)16>(int, int, int, T1 *, unsigned char *, float *, const float *, T
18.4%	1.841 s	677	2.719 ms	2.320 ms	518.078 µs	36.789 ms	2.556 ms	void cutlass::Kernel2<cutlass_80_tensorop_bf16_s16816gemm_relu_bf16_256x128_32x3_tn_align8>(T1::Params)
11.1%	1.115 s	261	4.272 ms	4.272 ms	4.267 ms	4.312 ms	4.796 µs	std::enable_if<!T7, void>::type internal::gemvx::kernel<int, int, __nv_bfloat16, __nv_bfloat16, __nv_bfloat16, float, (bool)0, (bc
7.7%	772.357 ms	196	3.941 ms	4.549 ms	1.634 ms	4.987 ms	1.042 ms	ampere_bf16_s16816gemm_bf16_256x128_ldg8_f2f_stages_32x3_tn
5.2%	519.840 ms	1568	331.530 µs	116.400 µs	8.704 µs	727.838 µs	319.168 µs	void kDequantizeBlockwise<__nv_bfloat16, (int)512, (int)64, (int)8, (int)2>(float *, unsigned char *, float *, T1 *, int, int)
3.3%	333.349 ms	7140	46.687 µs	46.688 µs	45.248 µs	48.320 µs	632 ns	void flash::flash_fwd_splitkv_kernel<Flash_fwd_kernel_traits<(int)128, (int)64, (int)128, (int)4, (bool)0, (bool)0, cutlass::bflo
2.6%	258.565 ms	168	1.539 ms	1.664 ms	526.558 µs	2.343 ms	579.114 µs	void flash::flash_fwd_splitkv_kernel<Flash_fwd_kernel_traits<(int)128, (int)64, (int)128, (int)4, (bool)0, (bool)0, cutlass::bflo
2.1%	210.829 ms	1120	188.240 µs	51.727 µs	2.143 µs	599.775 µs	242.483 µs	void at::native::elementwise_kernel<(int)128, (int)4, void at::native::gpu_kernel_impl_nocast<at::native::direct_copy_kernel_
1.8%	184.412 ms	7364	25.042 µs	3.936 µs	3.871 µs	991.710 µs	133.925 µs	void vllm::act_and_mul_kernel<c10::BFloat16, __nv_bfloat162, &vllm::silu_kernel<c10::BFloat16>, &vllm::packed_silu_kerne
1.0%	100.103 ms	196	510.731 µs	200.127 µs	27.776 µs	1.045 ms	464.623 µs	void kQuantizeBlockwise<__nv_bfloat16, (int)64, (int)2, (int)0, (int)2>(float *, T1 *, float *, unsigned char *, float *, int, int)
0.9%	93.917 ms	392	239.583 µs	159.823 µs	95.455 µs	523.551 µs	141.164 µs	ampere_bf16_s1688gemm_bf16_128x128_ldg8_f2f_stages_32x1_tn

46.8% of GPU time on quantization kernels.
kgemm_4bit_naive has no Tensor Core path — explains 15.2
→ 11.3 tok/s regression.



Decode blocks visibly wider than pure vLLM — kgemm_4bit_naive is slower than
Tensor Core gemvx path.

Potential Future Work

- ❖ Try AWQ instead of NF4 for quantization
- ❖ Can try speculative decoding with better models and stronger hardware.
- ❖ Fix kernel level analysis issues so that we have more comprehensive profiling

References

- ❖ **Kwon et al. (2023).** *Efficient Memory Management for Large Language Model Serving with PagedAttention*. Symposium on Operating Systems Principles (SOSP).
- ❖ **Bai et al. (2023).** *Qwen Technical Report*. arXiv.
- ❖ **Wolf et al. (2019).** *HuggingFace's Transformers: State-of-the-art Natural Language Processing*. arXiv.
- ❖ **Dettmers et al. (2023).** *QLoRA: Efficient Finetuning of Quantized LLMs*. arXiv.
- ❖ **Leviathan et al. (2023).** *Fast Inference from Transformers via Speculative Decoding*. International Conference on Machine Learning (ICML).
- ❖ **Lin et al. (2023).** *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration*. arXiv.

★ Thank you for
listening ★